

Escopo do Projeto Bot WhatsApp + IA

Objetivo Geral: Desenvolver um SaaS simples usando a WhatsApp Cloud API para coletar nome do usuário, confirmar termos de uso, salvar dados no MongoDB Atlas, e responder a interações com links fixos (Etapa 1) ou respostas de IA baseadas em leitura dinâmica de sites/PDFs (Etapa 2), mantendo compliance com políticas do WhatsApp e custos zero. O projeto será implantado na Secretaria de Cultura de Ponta Grossa-PR para promover difusão cultural, alinhado às ODS da ONU.

Funcionalidades-Chave:

- Coleta e confirmação de nome com loop até validação.
- Confirmação explícita de termos de uso (opt-in) para mensagens futuras, incluindo aceite de políticas de privacidade via link (ex: GitHub README).
- Menu de opções (Agenda, Edital, Falar com Atendente, Sair).
- Etapa 1: Envio de link fixo com base na opção escolhida.
- Etapa 2: Solicitação de dúvida, integração com IA (Groq com modelo llama-3.1-8b-instant) e resposta com leitura dinâmica de sites/PDFs.
- Salvamento de dados (phone, name) no MongoDB Atlas, com acceptedTerms para registrar opt-in.
- Gerenciamento de URLs via interface web nativa do MongoDB Atlas (visualização, edição e deleção).
- Compliance: Mensagem clara de consentimento e opção "SAIR" para cancelar.

Restrições:

- Custos zero: Usar tiers gratuitos (WhatsApp Cloud API, MongoDB Atlas, AWS Lambda com API Gateway, IA Groq, axios + cheerio).
- Conformidade com políticas do WhatsApp (opt-in explícito, templates para mensagens proativas).
- Mudanças mínimas no código existente (app.js), preservando estrutura, nomes e aceite de termos de uso.
- Estados em memória (conversationStates) por simplicidade; migração para DB só se escalar.

Dependências:

- Conta no Meta Business Manager (business.facebook.com) – a Secretaria precisará criar uma conta gratuita, verificar empresa, e gerar token permanente para System User com permissões whatsapp_business_messaging e whatsapp_business_management.
- Número de telefone dedicado (SMS para verificação) – vinculado à conta Meta.
- Servidor no AWS Lambda (free tier) com API Gateway – a Secretaria criará conta AWS gratuita, configurando Lambda (runtime Node.js 20.x) e Gateway para webhook.
- MongoDB Atlas (free tier, 512MB) – conta gratuita na MongoDB, com cluster M0, database whatsapp-users, e interface web para gerenciamento de dados.
- Groq API key (free tier) – conta gratuita no Groq para chave de API, integrada via groq-sdk.
- Dependências Node.js: express, axios, cheerio, dotenv, mongoose, groq-sdk, pdf2json, aws-serverless-express.

Etapa 1: A ser Implementada

Fluxo: Coleta nome, confirmação, termos, menu com links fixos. Código app.js será configurado com webhook, estados em memória, salvamento no MongoDB. Deploy no AWS Lambda (URL gerada via API Gateway). Testes manuais via Postman e WhatsApp serão realizados, com salvamento de dados.

Avaliação das ODS - Etapa 1

- ODS 4 - Meta 4.7: "Até 2030, garantir que todos os alunos adquiram os conhecimentos e habilidades necessárias para promover o desenvolvimento sustentável, incluindo, entre outros,

por meio da educação para o desenvolvimento sustentável e estilos de vida sustentáveis, direitos humanos, igualdade de gênero, promoção de uma cultura de paz e não violência, cidadania global e valorização da diversidade cultural e da contribuição da cultura para o desenvolvimento sustentável." Atendida pela divulgação de editais e agendas culturais via bot WhatsApp, promovendo acesso sustentável à informação cultural. Fonte: <https://brasil.un.org/pt-br/sdgs/4>

- ODS 9 - Meta 9.c: "Aumentar significativamente o acesso às tecnologias de informação e comunicação e se esforçar para fornecer acesso universal e a preços acessíveis à internet nos países menos desenvolvidos, até 2020." Atendida ao usar WhatsApp gratuito para consultas culturais, aumentando inclusão digital. Fonte: <https://brasil.un.org/pt-br/sdgs/9>
- ODS 11 - Meta 11.4: "Reforçar esforços para proteger e salvaguardar o patrimônio cultural e natural do mundo." Atendida facilitando acesso a agendas culturais, preservando patrimônio via tecnologia. Fonte: <https://brasil.un.org/pt-br/sdgs/11>
- ODS 17 - Meta 17.16: "Reforçar a parceria global para o desenvolvimento sustentável, complementada por parcerias multisetoriais que mobilizem e compartilhem conhecimento, expertise, tecnologia e recursos financeiros, a fim de apoiar a consecução dos objetivos de desenvolvimento sustentável em todos os países, em particular os países em desenvolvimento." Atendida pela parceria universidade-secretaria de cultura, compartilhando tecnologia para resolver dúvidas comunitárias sobre editais e agendas. Fonte: <https://brasil.un.org/pt-br/sdgs/17>

Etapas 2: A ser Implementada

Nesta etapa, substitui links por consultas à IA com contexto de sites/PDFs oficiais, respondendo dúvidas sobre editais e agendas de forma inteligente. Será aplicado IA Groq (llama-3.1-8b-instant) e axios + cheerio.

Funcionalidades-Chave:

- Coleta e confirmação de nome com loop até validação.
- Confirmação explícita de termos de uso (opt-in) – aceito ao digitar "1", incluindo aceite de políticas de privacidade.
- Menu de opções (Agenda, Edital, Falar com Atendente, Sair).
- Scraping dos sites oficiais com axios + cheerio.
- IA Groq (llama-3.1-8b-instant) responde dúvidas com base no contexto do site/PDF (extração via pdf2json, busca fuzzy).
- Cache de scraping (5 min) + retry (3 tentativas).
- Quota diária (200 chamadas em memória).
- Salvamento no MongoDB Atlas: apenas phone e name.
- Compliance: política de privacidade, opção "SAIR".

Restrições:

- Custos zero: WhatsApp Cloud API (1.000 conv/mês), MongoDB Atlas free tier, AWS Lambda free tier, Groq API gratuita (ilimitado).
- Conformidade com políticas do WhatsApp (opt-in explícito).
- Mudanças mínimas no código existente (app.js).
- Estados em memória (conversationStates).

Dependências:

- express, axios, cheerio, dotenv, mongoose, groq-sdk, pdf2json, aws-serverless-express (instalado via npm install).

- Variáveis de ambiente: GROQ_API_KEY, MONGODB_URI, ACCESS_TOKEN, PHONE_NUMBER_ID, VERIFY_TOKEN e PORT.

Avaliação das ODS - Etapa 2 (Integração com IA para Respostas a Dúvidas)

As ODS da Etapa 1 permanecem atendidas, com adicionais:

- ODS 8 - Meta 8.2: "Atingir níveis mais elevados de produtividade das economias através da diversificação, modernização tecnológica e inovação, nomeadamente através da concentração em setores de maior valor acrescentado e grande densidade de mão-de-obra." Atendida pois a IA otimiza respostas culturais, aumentando produtividade da secretaria via modernização tecnológica, reduzindo consultas manuais. Fonte: <https://brasil.un.org/pt-br/sdgs/8>
- ODS 9 - Meta 9.5: "Fortalecer a investigação científica, melhorar as capacidades tecnológicas de setores industriais em todos os países, particularmente os países em desenvolvimento, inclusive, até 2030, incentivando a inovação e aumentando substancialmente o número de trabalhadores de pesquisa e desenvolvimento por milhão de pessoas e os gastos público e privado em pesquisa e desenvolvimento." Atendida ao integrar IA para inovação em serviços culturais, fortalecendo capacidades tecnológicas locais para respostas contextualizadas. Fonte: <https://brasil.un.org/pt-br/sdgs/9>
- ODS 9 - Meta 9.b: "Apoiar o desenvolvimento tecnológico, a pesquisa e a inovação nacionais nos países em desenvolvimento, inclusive garantindo um ambiente político propício para, inter alia, a diversificação industrial e a agregação de valor às commodities." Atendida apoiando inovação nacional via IA gratuita, diversificando acesso a informações culturais em contextos em desenvolvimento. Fonte: <https://brasil.un.org/pt-br/sdgs/9>

Limites do Free Tier AWS (válidos em 2025, com base em dados oficiais)

- **AWS Lambda:** 1 milhão de invocações gratuitas/mês + 400.000 GB-segundos de computação (ex: com 1024 MB memória, ~400 segundos gratuitos/mês). Cada chamada no bot (GET/POST webhook) é uma invocação curta (~1-5s), totalizando <100 invocações/mês no seu caso – bem abaixo do limite.
- **API Gateway:** 1 milhão de chamadas HTTP gratuitas/mês. Cada interação WhatsApp gera 1-2 requests – total <100/mês, zero custo.
- **Outros:** Sem dados armazenados na AWS (usamos MongoDB Atlas free), sem transferências extras. Total: 0 custo se abaixo desses limites.

Como saber/monitorar:

- Após criar conta, acesse o Billing Dashboard (aws.amazon.com > Minha Conta > Billing). Veja "Free Tier Usage" para rastrear consumo mensal.
- Configure alertas grátis: No Billing > Preferences > Billing Alerts > Ative "Receive Free Tier Usage Alerts". Defina notificações por e-mail para 80% de uso (ex: via CloudWatch Alarms, free tier).
- Relatórios: Baixe CSV mensal em Billing > Bills para auditoria. Se ultrapassar (improvável com 66 chamadas/mês), custo seria ~R\$0,001/invocação extra – mas monitore semanalmente.

Criação de Conta AWS: Explicação Detalhada

Para implantar o bot na AWS (necessário para estabilidade no webhook WhatsApp, evitando timeouts em alternativas como Render e Vercel), é preciso criar uma conta gratuita. **Sim, requer cartão de crédito para verificação de identidade – é padrão global da AWS para prevenir fraudes, mesmo no Free Tier.** Não há cobrança se o uso ficar dentro dos limites (como no nosso caso). Explico passo a passo, riscos e como evitar custos – isso é essencial para compliance no projeto de extensão (zero remuneração, foco comunitário).

Por que cartão? A AWS cobra uma taxa temporária de US\$1 (devolvida em 3-5 dias) para validar o cartão real. É como uma "autorização" bancária (não débito), similar a hotéis. Após, se uso exceder Free Tier (ex: >1M invocações), cobra automaticamente – mas configuraremos alertas para evitar.

Passos para Criar Conta (Tempo: ~10min, Gratuito):

1. Acessar https://aws.amazon.com/pt_br/ e clique em "Criar uma conta gratuita".
2. Preencher e-mail institucional (ex: cultura@pontagrossa.pr.gov.br) e senha. Selecionar "Conta Profissional" (para entidade pública).
3. Informe dados da Secretaria: Nome (ex: "Secretaria de Cultura de Ponta Grossa"), endereço oficial, telefone. Preferencialmente não usar dados pessoais.
4. Na etapa de pagamento: Insira cartão de crédito válido (da Secretaria ou autorizado). A AWS autoriza US\$1 (R\$5-6) temporariamente – verificar extrato, mas será estornado automaticamente em 1 ou 2 dias.
5. Verifique identidade: A AWS envia código por SMS/e-mail; insira para ativar.
6. Escolha "Suporte Básico" (gratuito). Conta ativada!

Riscos e Como Evitar Cobranças (Importante!):

- **Risco Principal:** Se uso ultrapassar Free Tier (ex: >1M chamadas/mês devido a spam ou erro), cobra ~R\$0,001/invocação. No nosso projeto: Com 2-3 chamadas/dia útil (44-66/mês), risco quase zero – equivale a <0,01% do limite.
- **Evitando Cobranças:**
 - Monitorar semanalmente no Billing Dashboard (acesso gratuito).
 - Ativar alertas: Após login, vá em Billing > Preferences > Ative "Free Tier Usage Alerts" e configure e-mail para notificação a 80% de uso.
 - Definir Budget Alert grátis: Em Billing > Budgets > Crie budget mensal de US\$0 (alerta se >US\$0). Recebe e-mails se aproximar.
 - Pausa/Apaga Recursos: Se inativo, delete Lambda/Gateway via console (sem custo).
 - Testes: Use apenas para o bot – evitar outros serviços AWS.
- **Sem Cartão Alternativo?** AWS exige para novas contas; não aceita boleto/pré-pago. Sugestão usar cartão corporativo ou testar inicial com conta pessoal (transferir depois via AWS Organizations, free).
- **Benefícios para Secretaria:** Zero custo real, escalável, estabilidade (sem timeouts como em Render ou Vercel). Alinha à ODS 9.c (inclusão digital via tech gratuita).

Documentação no Relatório do Projeto:

- Criar seção "Gestão de Custos e Monitoramento AWS" no relatório final/PDCA (fase Plan/Check):
 - Estimativa: "Uso esperado: 2-3 chamadas/dia útil x 22 dias = 44-66 invocações/mês. Limites Free Tier: 1M invocações (Lambda) + 1M requests (Gateway) – 0% utilizado, custo R\$0."
 - Evidências: Prints de Billing Dashboard inicial (US\$0), configuração de alertas.
 - Responsabilidades: "Diretoria monitora mensalmente; desenvolvedor auxilia setup inicial."

Fontes: https://aws.amazon.com/pt_br/free/ (detalhes Free Tier) e https://docs.aws.amazon.com/pt_br/awsaccountbilling/latest/aboutv2/billing-getting-started.html (guia billing).

=====

Documentação do Setup

Passo a Passo para Criação do AWS Lambda e API Gateway

Para implantar o webhook do bot WhatsApp na AWS (free tier), siga esses passos na console AWS (aws.amazon.com, região sa-east-1 para baixa latência no Brasil). Isso garante escalabilidade sem custos para uso baixo (~66 invocações/mês). Monitore no Billing Dashboard para confirmar zero cobrança.

1. Crie uma Conta AWS (se não tiver)

- Acesse https://aws.amazon.com/pt_br/ e clique em "Criar uma conta AWS gratuita".
- Preencha e-mail, senha e dados da entidade (ex: Secretaria de Cultura de Ponta Grossa-PR).
- Insira cartão de crédito para validação (autorização temporária de US\$1, estornada em 3-5 dias).
- Verifique identidade via SMS/e-mail.
- Fonte: https://docs.aws.amazon.com/pt_br/awsaccountbilling/latest/aboutv2/billing-getting-started.html

2. Crie a Função Lambda

- Vá em Serviços > Lambda > Criar função.
- Selecione "Criar do zero".
- Nome: bot-whatsapp.
- Runtime: Node.js 20.x.
- Arquitetura: x86_64.
- Clique em "Criar função".
- No código: Compacte app.js e dependências (node_modules via npm i --production) em ZIP (sem .env).
- Upload ZIP via "Código" > "Upload de .zip".
- Handler: app.handler (configurado via aws-serverless-express).
- Configurações avançadas: Memória 1024 MB, timeout 120s.
- Variáveis de ambiente: Adicione MONGODB_URI, GROQ_API_KEY, PHONE_NUMBER_ID, ACCESS_TOKEN, VERIFY_TOKEN=meuTokenSecreto2025.
- Fonte: https://docs.aws.amazon.com/pt_br/lambda/latest/dg/gettingstarted.html

3. Crie a API Gateway

- Vá em Serviços > API Gateway > Criar API > REST API > Construir.
- Nome: bot-whatsapp-api.
- Tipo: Regional.
- Clique em "Criar API".
- Recursos: Crie /webhook.
- Métodos: ANY (para GET/POST).
- Integração: Lambda Proxy, selecione a função bot-whatsapp.
- Autorização: NONE.
- Timeout de integração: 29s (default).
- Implante: Ações > Implantar API > Estágio default.
- URL gerada: https://[id].execute-api.sa-east-1.amazonaws.com/default/webhook.
- Fonte: https://docs.aws.amazon.com/pt_br/apigateway/latest/developerguide/welcome.html

4. Configure Permissões

- Na Lambda > Permissões > Adicionar permissões > Função IAM role > Anexe política para invocar via Gateway (gerada automaticamente ao integrar).
- Teste: No Meta Developers, configure webhook com a URL e token.

5. Monitoramento de Custos

- Ative alertas: Billing > Preferências > Alertas de faturamento > Ativar.
- Crie orçamento: Billing > Orçamentos > Criar (US\$0/mês, alerta a 80%).
- Verifique uso: Billing > Free Tier Usage.
- Fonte: https://docs.aws.amazon.com/pt_br/awsaccountbilling/latest/aboutv2/free-tier-limits.html

=====

Passo a Passo para Configurar MongoDB Atlas (Free Tier M0)

Para o projeto de extensão, use o free tier do MongoDB Atlas (512MB, sem custos). Foque em criar um cluster, usuário/senha e liberar IPs públicos (0.0.0.0/0 temporário para IPs dinâmicos da AWS).

Tempo estimado: 10-15min.

1. Crie uma Conta no MongoDB Atlas

- Acesse <https://account.mongodb.com/register> e preencha e-mail institucional (ex: cultura@pontagrossa.pr.gov.br), senha e nome da organização (ex: Secretaria de Cultura Ponta Grossa-PR).
- Verifique o e-mail recebido para ativar a conta.
- Fonte: <https://www.mongodb.com/docs/atlas/getting-started/#std-label-get-started-create-account>

2. Crie um Cluster Free Tier

- Após login, clique em "Build a Database" > "M0 Free" (shared cluster).
- Selecione provedor AWS, região sa-east-1 (para baixa latência no Brasil).
- Nome do cluster: ex: ClusterBotWhatsapp.
- Clique em "Create" (gratuito, sem cartão necessário).
- Fonte: <https://www.mongodb.com/docs/atlas/tutorial/deploy-free-tier-cluster/>

3. Crie Usuário e Senha (Database User)

- No dashboard do cluster > "Database Access" > "Add New Database User".
- Método: Password (Username and Password).
- Username: ex: bot_user (evite nomes genéricos como "admin").
- Password: Gere uma senha forte (ex: combine letras, números, símbolos; anote em local seguro).
- Database User Privileges: Selecione "Atlas Admin" (readWriteAnyDatabase para acesso total no free tier).
- Clique em "Add User".
- Fonte: <https://www.mongodb.com/docs/atlas/security-add-mongodb-users/>

4. Libere IPs (Network Access: Allow 0.0.0.0/0)

- No dashboard > "Network Access" > "Add IP Address".
- IP Address: 0.0.0.0/0 (allow from anywhere – temporário para IPs dinâmicos da Lambda/AWS).
- Comment: ex: "Acesso temporário para webhook WhatsApp".
- Clique em "Add IP Address".
- Nota: Para produção, restrinja a IPs específicos; aqui, mantém zero custos e simplicidade.
- Fonte: <https://www.mongodb.com/docs/atlas/security/ip-access-list/>

5. Obtenha URI de Conexão

- No dashboard do cluster > "Connect" > "Connect your application".

- Driver: Node.js (versão 4.0 ou superior).
- Copie a URI:
mongodb+srv://<username>:<password>@clusterbotwhatsapp.y2xowl.mongodb.net/?retryWrites=true&w=majority (substitua <username> e <password>).
- Adicione ao .env como MONGODB_URI.

Teste conexão via MongoDB Compass (gratuito) ou código local. Documente no relatório: prints de cada passo para compliance. Fonte geral: <https://www.mongodb.com/docs/atlas/>

Observações:

No MongoDB Atlas com Mongoose, os databases e collections são criados automaticamente pela aplicação ao primeiro uso de um modelo (ex: ao executar User.findOne() ou Edital.create()). Não é necessário criá-los manualmente na interface web do Atlas.

- O database é definido na URI de conexão (ex: mongodb+srv://.../whatsapp-users?...), e o Mongoose o cria na primeira operação.
- As collections seguem o nome do modelo: 'users' para User (pluralizado automático) e 'editais' para Edital (especificado explicitamente no schema).

Para testar, rode a app localmente após configurar a URI em .env – o Atlas gerará tudo na primeira query. Fonte: <https://mongoosejs.com/docs/connections.html#databases-and-collections>.

=====

Passo a Passo para Configurar WhatsApp Cloud API no Meta

Para o projeto de extensão, a Secretaria de Cultura de Ponta Grossa-PR deve criar contas gratuitas no Meta. Foque em compliance (opt-in, privacidade) e zero custos (API gratuita até 1.000 conversas/mês). Tempo estimado: 1-3 dias (devido à aprovação de documentos). Use e-mail institucional.

1. Crie uma Conta no Meta Business Manager

- Acesse <https://business.facebook.com> e clique em "Criar conta".
- Preencha nome da empresa (ex: "Secretaria de Cultura Ponta Grossa"), e-mail institucional, endereço oficial.
- Confirme via e-mail/SMS.
- Fonte: <https://business.facebook.com/create>

2. Verifique a Empresa (Envio de Documentos)

- No Business Manager > Configurações > Informações da Empresa > Verificação.
- Envie documentos: CNPJ da Prefeitura/Secretaria, comprovante de endereço (oficial), e ID do responsável (RG/CPF).
- Aguarde aprovação (1-2 dias úteis; Meta notifica por e-mail).
- Fonte: <https://www.facebook.com/business/help/159334372090593>

3. Crie o App como Desenvolvedor

- Acesse <https://developers.facebook.com> e logue com a conta Business.
- Clique em "Meus Apps" > "Criar App".
- Tipo: "Business" (para WhatsApp).
- Nome: ex: "Bot Cultura PG".
- Vincule à conta Business aprovada.

- Fonte: <https://developers.facebook.com/docs/development/create-an-app>

4. Adicione WhatsApp como Produto

- No dashboard do app > "Adicionar Produto" > Selecione "WhatsApp".
- Configure API versão 24.0 (default atual).
- Fonte: <https://developers.facebook.com/docs/whatsapp/cloud-api/get-started>

5. Adicione Número de Telefone

- No dashboard WhatsApp > "Configurações" > "Adicionar Número".
- Insira número dedicado (não usado em app pessoal; verifique via SMS/PIN).
- Confirme propriedade (chamada/SMS).
- Fonte: <https://developers.facebook.com/docs/whatsapp/cloud-api/phone-numbers>

6. Crie System User para a Aplicação

- No Business Manager > Usuários > Usuários do Sistema > "Adicionar".
- Nome: ex: "Bot User".
- Função: "Admin".
- Ativos: Atribua o app e WhatsApp.
- Fonte: <https://www.facebook.com/business/help/1492627900875762>

7. Gere Token Permanente e Libere Permissões

- No dashboard do app > WhatsApp > "Configurações" > "API Setup".
- Selecione System User criado > Gere token permanente.
- Permissões: whatsapp_business_messaging (para envio/recebimento), whatsapp_business_management (gerenciamento).
- Versão API: Confirme v24.0.
- Fonte: <https://developers.facebook.com/docs/graph-api/reference/v24.0>

8. Cadastre o Webhook

- No dashboard WhatsApp > "Configurações" > "Webhooks" > "Adicionar".
- URL: ex: [https://\[id\].execute-api.sa-east-1.amazonaws.com/default/webhook](https://[id].execute-api.sa-east-1.amazonaws.com/default/webhook).
- Token de Verificação: ex: "meuTokenSecreto2025".
- Campos: Selecione "messages" e "statuses".
- Teste: Envie mensagem de teste via painel.
- Fonte: <https://developers.facebook.com/docs/whatsapp/cloud-api/webhooks/configure-webhooks>

9. Outros Itens Essenciais (Não Esqueça)

- Teste Inicial: Use "From Sample" no painel WhatsApp para simular mensagens.
- Compliance: Configure política de privacidade (link no bot) e opt-in explícito no código.
- Monitoramento: No Business Manager > Insights para uso (garante <1.000 conv/mês gratuito).
- Erros Comuns: Se token expirar, regere; verifique versão API v24.0 em chamadas Axios.

Fonte geral: <https://developers.facebook.com/docs/whatsapp/cloud-api>

=====

Passo a Passo para Criar Conta e Token na Groq API

Para integrar a IA Groq (modelo llama-3.1-8b-instant) na Etapa 2 do projeto de extensão, siga esses passos para criar uma conta gratuita (zero custos, com rate limits adequados ao uso baixo ~200 calls/dia). A API é free tier, alinhada ao escopo de zero remuneração e conformidade.

1. Crie uma Conta no Groq:

- Acessar <https://console.groq.com/login>.
- Clicar em "Sign Up" (ou "Register").
- Preencher e-mail institucional (ex: cultura@pontagrossa.pr.gov.br), senha e nome (use dados da Secretaria para compliance).
- Confirmar via e-mail recebido (clique no link de verificação).

2. Gere a API Key:

- Após login, vá para o dashboard > "API Keys" (ou diretamente <https://console.groq.com/keys>).
- Clicar em "Create API Key".
- Nomear a key (ex: "BotCulturaPG").
- Copiar a key gerada (formato: gsk_...).
- Armazenar em local seguro; adicione ao .env como GROQ_API_KEY.

3. Integre no Código:

- No app.js, confirme a inicialização: `const groq = new Groq({ apiKey: process.env.GROQ_API_KEY });`.
- Testar localmente com uma chamada simples via Postman ou código de teste.

Fonte: <https://console.groq.com/docs/quickstart> (guia oficial para setup e autenticação). Monitorar rate limits no dashboard para evitar excedentes (projeto usa quota interna de 200 calls/dia para escalabilidade).